

NYC TAXI AWS DATA ENGINEERING PROJECT

Complete End-to-End Implementation with Airflow 3.x, PySpark, Glue & Redshift

1. PROJECT OVERVIEW

This project implements a production-grade **Medallion Architecture** (Bronze → Silver → Gold) data pipeline for NYC Yellow Taxi trip data entirely on AWS. The pipeline is fully orchestrated using **Apache Airflow 3.2.2** running in Docker, with **PySpark** for transformations, **AWS Glue** for cataloging, and **Amazon Redshift Serverless** as the final analytics warehouse.

Key Achievements: Parameterized monthly ingestion, comprehensive data quality framework, idempotent Redshift loading, full Glue catalog automation, and clean separation of concerns across all layers.

2. SYSTEM ARCHITECTURE

The architecture follows a clear layered approach with strong orchestration:

- Orchestration Layer** — Airflow 3.x DAG controls the entire flow (Bronze → Silver → Gold → Glue/Redshift)
- Bronze Layer** — Raw Parquet ingestion from NYC TLC CloudFront into S3 (Hive partitioned by year/month)
- Silver Layer** — PySpark cleaning, type casting, derived columns + data quality validation
- Gold Layer** — Business aggregations (daily_summary + hourly_demand)
- Governance Layer** — AWS Glue Crawlers + Catalog for all three layers
- Analytics Warehouse** — Amazon Redshift Serverless (final Gold data loaded via COPY after Glue)

3. TECHNOLOGY STACK

Layer	Technology	Role
Orchestration	Apache Airflow 3.2.2 (Docker + Redis, Celery)	DAG orchestration, parameterization, task dependencies
Bronze Ingestion	boto3 + CloudFront	Download from public NYC TLC → S3 upload (idempotent)
Silver / Gold	PySpark 3.5 + Hadoop AWS	Cleaning, transformations, aggregations
Data Quality	Custom PySpark framework	Null/NaN validation, range checks, reporting
Catalog & Governance	AWS Glue + Crawlers	Auto-cataloging Bronze/Silver/Gold tables
Analytics Warehouse	Amazon Redshift Serverless	Final Gold tables via COPY (idempotent)
Infrastructure	Docker + docker-compose	Reproducible Airflow 3.x

		environment
Configuration	configparser + AWS credentials	Secure credential and parameter management

4. ORCHESTRATION LAYER — Airflow 3.x DAG

DAG: `nyc_taxi_data_pipeline`` (defined in `pipelines/nyc_taxi_ingestion.py``)

This is the **central orchestrator** that drives the entire pipeline execution in the correct order.

- Task Flow (Linear + Parallel Branches):**
 - bronze_ingestion → silver_transformation → data_quality_check → gold_aggregation
 - Parallel branch (after Gold): setup_glue_infrastructure → run_glue_crawlers
 - Parallel branch (after Gold): setup_redshift_warehouse → load_redshift_warehouse
 - Final dependency ensures Glue and Redshift only run after Gold layer completes successfully
- Airflow 3.x Modern Features Used:**
 - New `@dag`` and `@task`` decorators from `airflow.sdk``
 - Parameterized DAG with `params`` (year/month)
 - TaskFlow API with automatic XCom passing between tasks
 - Dockerized setup with CeleryExecutor, Redis, and Postgres

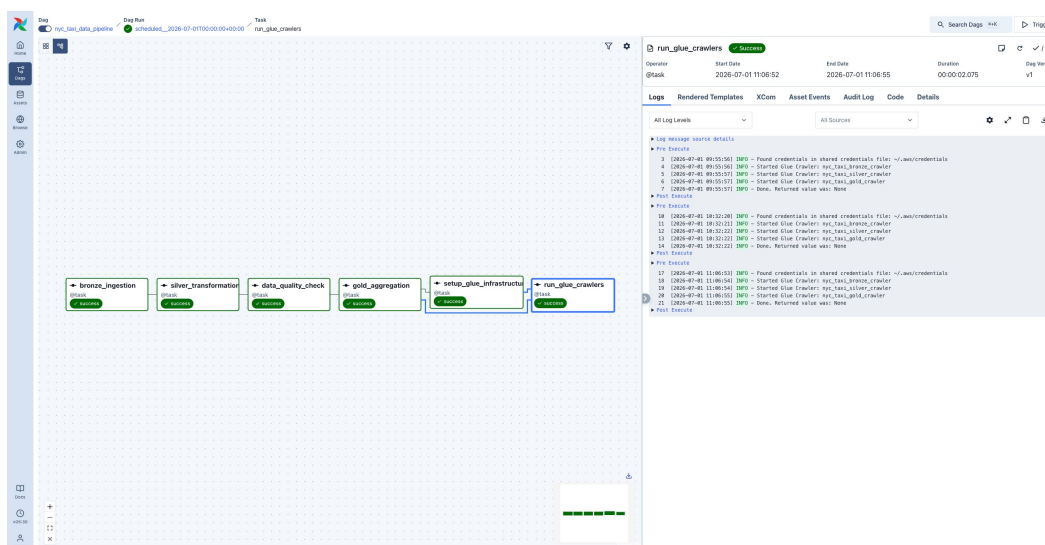


Figure 1: Airflow 3.x DAG Graph View — Complete pipeline showing all tasks with successful execution status

Name	Container ID	Image	Port(s)	CPU (%)	Memory usage	Memory (%)	Disk read/write	Network I/O	PIDs	Last started	Actions
10nyc_taxi_aws_data_engineering	7f8845c926d	-	-	3.66%	1.26GB / 46.49GB	16.31%	132.72MB / 252.1	769.4KB / 689.2B	6, 0, 10, 3, 6, 2, 2	18 seconds ago	[Stop] [Refresh] [Delete]
redis-1	7f8845c926d	redis:7	6379:6379	0.4%	22.19MB / 7.75G	0.28%	13.4MB / 0B	22.4KB / 12.4KB	6	22 seconds ago	[Stop] [Refresh] [Delete]
airflow-init-1	fcfb21f80dc	myc-taxi-airflow:latest	-	0%	0B / 0B	0%	0B / 0B	0B / 0B	0	22 seconds ago	[Stop] [Refresh] [Delete]
postgres-1	e9b1e9a51943	postgres:16	5432:5432	0.7%	80.84MB / 7.75G	1.02%	47MB / 57.7MB	470KB / 299KB	10	22 seconds ago	[Stop] [Refresh] [Delete]
airflow-dag-processor-1	a1aa1397dc64	myc-taxi-airflow:latest	-	0.62%	167.9MB / 7.75G	2.12%	25.5MB / 52.6MB	78.9KB / 103KB	3	18 seconds ago	[Stop] [Refresh] [Delete]
airflow-api-server-1	9f178876e874	myc-taxi-airflow:latest	8080:8080	0.11%	205.6MB / 7.75G	2.59%	4.33MB / 36.1MB	14.6KB / 21.2KB	6	18 seconds ago	[Stop] [Refresh] [Delete]
airflow-scheduler-1	456fe53970f	myc-taxi-airflow:latest	-	1.39%	145.6MB / 7.75G	1.84%	295KB / 32.1MB	164KB / 227KB	2	18 seconds ago	[Stop] [Refresh] [Delete]
airflow-worker-1	55d34517af5a	myc-taxi-airflow:latest	-	0.44%	671.3MB / 7.75G	8.46%	42.2MB / 74.1MB	19.5KB / 26.6KB	24	18 seconds ago	[Stop] [Refresh] [Delete]

Figure 2: Docker Desktop — All Airflow 3.x containers running (postgres, redis, api-server, scheduler, worker, dag-processor)

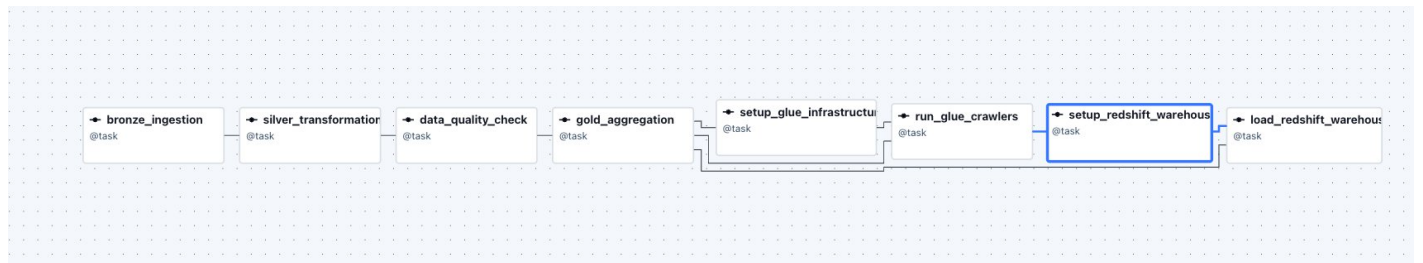


Figure 3: Docker Desktop — Airflow 3.x DAG Graph View — Complete pipeline showing all tasks pre execution status

5. BRONZE LAYER — Raw Data Ingestion

Purpose: Ingest raw NYC Yellow Taxi trip records from the public NYC TLC CloudFront distribution into S3 with proper Hive-style partitioning (`year=YYYY/month=MM`). This is the **first task** executed by the Airflow DAG.

- Implementation Details:**
 - `etls/aws\_etl.py` downloads Parquet files from `https://d37ci6vzurychx.cloudfront.net/trip-data/`
 - Uploads to bucket `s3://himanshu-nyc-taxi-bronze/bronze/nyc\_taxi/year=YYYY/month=MM/`
 - Idempotency check using `head\_object` before download (skips if exists)
 - Parameterized by `year` and `month` via Airflow DAG `params`

Name	Type	Last modified	Size	Storage class
athens_query_workspace/	Folder	-	-	-
bronze/	Folder	-	-	-
gold/	Folder	-	-	-
silver/	Folder	-	-	-

Figure 4: Amazon S3 Bucket Structure — himanshu-nyc-taxi-bronze showing bronze/, silver/, gold/ folders

6. SILVER LAYER — Cleaning & Transformation

Purpose: Clean raw data, enforce consistent schema through explicit casting, derive useful analytical columns, and run comprehensive data quality checks before aggregation. This is the **second task** in the DAG.

- **Key Transformations:**
 - Explicit casting of all columns for schema stability (IntegerType, DoubleType, TimestampType)
 - Derived columns: `pickup\_date`, `pickup\_year`, `pickup\_month`, `pickup\_dayofweek`, `pickup\_hour`, `trip\_duration\_minutes`
 - Basic filters: `passenger_count > 0`, `trip_distance > 0`, `total_amount > 0`
- **Data Quality Framework:**
 - Layer-aware null/NaN detection (handles numeric vs non-numeric columns correctly)
 - Range validation for negative amounts, zero/negative distances, and negative durations
 - Returns structured quality report with status `PASSED` or `WARNINGS`

7. GOLD LAYER — Business Aggregations

Purpose: Create analytics-ready aggregated tables for BI and reporting use cases. This is the **final transformation task** in the DAG before Glue and Redshift branches.

- **Two Gold Datasets:**
 - `daily\_summary`: total_trips, total_revenue, avg_trip_duration_min, avg_tip_amount, avg_passengers (grouped by year/month/dayofweek)
 - `hourly\_demand`: Same metrics + `pickup\_hour` for finer time-based analysis
 - Both partitioned by `pickup\_year` and `pickup\_month` for efficient querying

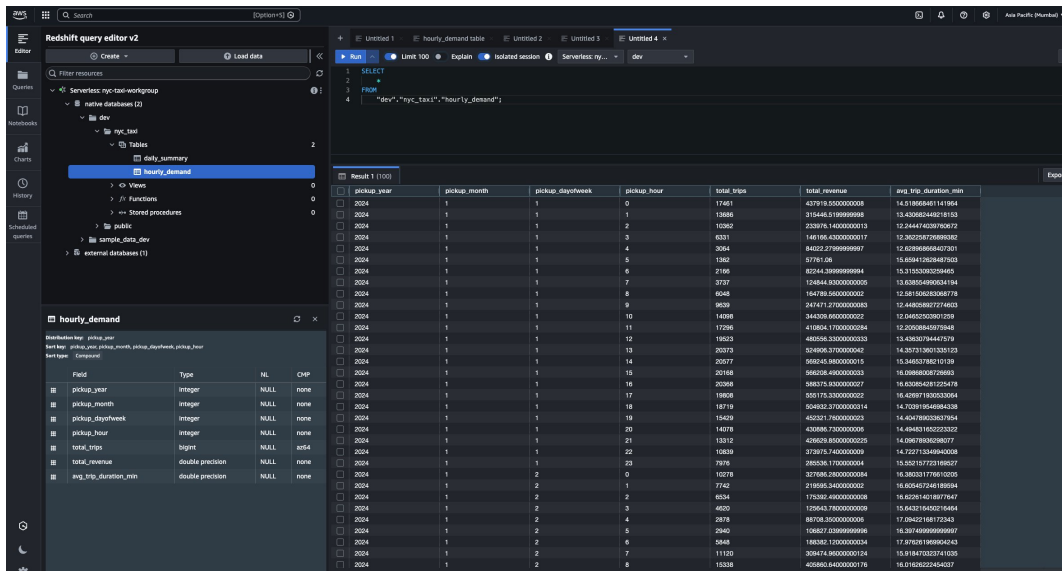


Figure 5: Amazon Redshift Query Editor — hourly_demand table showing aggregated data for January 2024

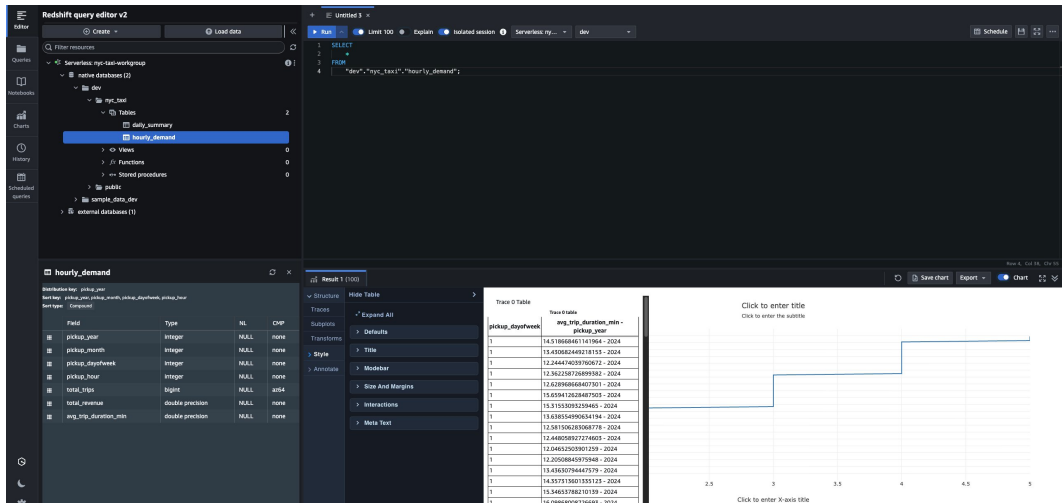


Figure 6: Redshift Query Results — Line chart visualization of average trip duration by day of week

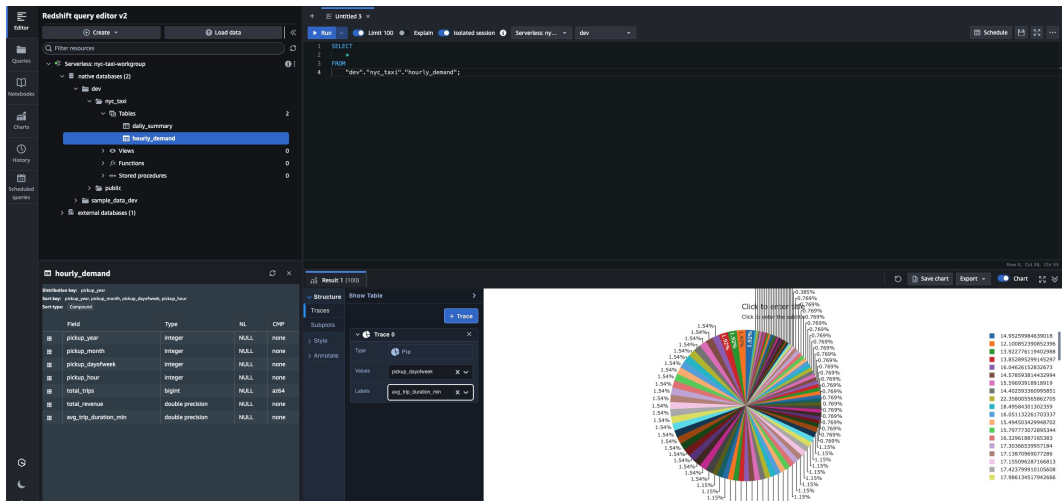
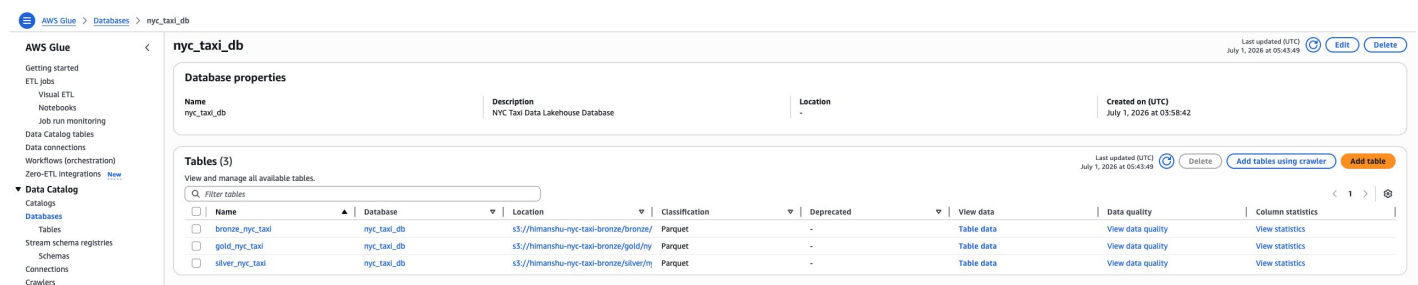


Figure 7: Redshift Query Results — Pie chart visualization of trip distribution by day of week

8. GLUE CATALOG & GOVERNANCE (Triggered after Gold)

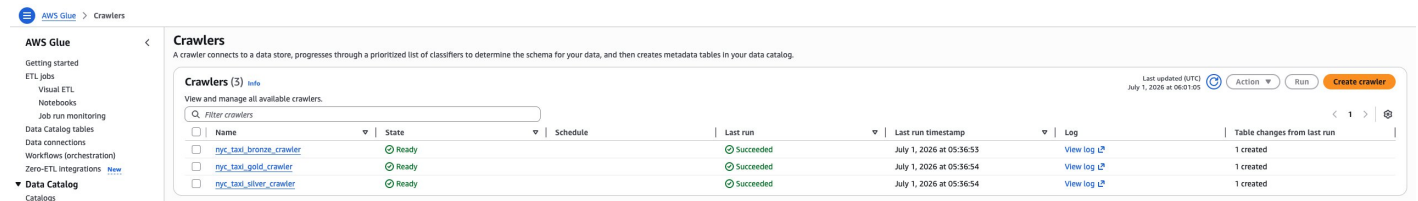
- ****AWS Glue Implementation:****
 - Database created: `nyc\_taxi\_db`
 - Three crawlers configured with table prefixes: `bronze\_`, `silver\_`, `gold\_`
 - Crawlers run ****after Gold layer**** to keep catalog automatically in sync with S3 data
 - IAM role `AWSGlueServiceRole-nyc-taxi` with proper S3 permissions



The screenshot shows the AWS Glue console for the database `nyc_taxi_db`. The database properties are listed, and a table of tables is shown below. The table has columns for Name, Database, Location, Classification, Deprecated, View data, Data quality, and Column statistics. Three tables are listed: `bronze_nyc_taxi`, `gold_nyc_taxi`, and `silver_nyc_taxi`, all located in `s3://himanshu-nyc-taxi-bronze/gold/ny` and classified as `Parquet`.

Name	Database	Location	Classification	Deprecated	View data	Data quality	Column statistics
<code>bronze_nyc_taxi</code>	<code>nyc_taxi_db</code>	<code>s3://himanshu-nyc-taxi-bronze/bronze/ny</code>	<code>Parquet</code>	-	Table data	View data quality	View statistics
<code>gold_nyc_taxi</code>	<code>nyc_taxi_db</code>	<code>s3://himanshu-nyc-taxi-bronze/gold/ny</code>	<code>Parquet</code>	-	Table data	View data quality	View statistics
<code>silver_nyc_taxi</code>	<code>nyc_taxi_db</code>	<code>s3://himanshu-nyc-taxi-bronze/silver/ny</code>	<code>Parquet</code>	-	Table data	View data quality	View statistics

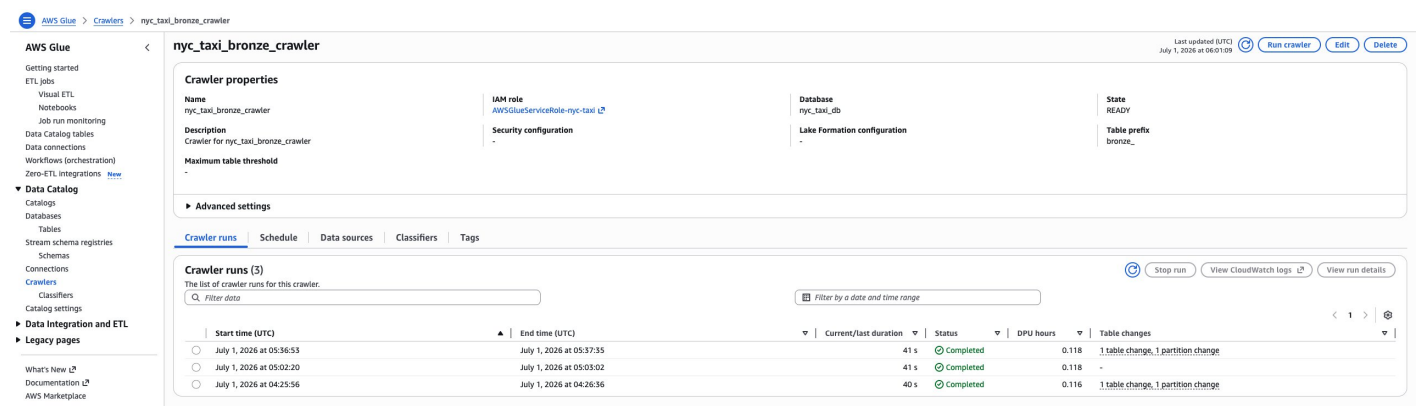
Figure 8: AWS Glue Database — `nyc_taxi_db` showing three tables (`bronze_nyc_taxi`, `silver_nyc_taxi`, `gold_nyc_taxi`)



The screenshot shows the AWS Glue console for the crawlers. The crawlers are listed in a table with columns for Name, State, Schedule, Last run, Last run timestamp, Log, and Table changes from last run. All three crawlers are in a `Ready` state and have successfully run.

Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last run
<code>nyc_taxi_bronze_crawler</code>	<code>Ready</code>	-		July 1, 2026 at 05:36:53	View log L2	1 created
<code>nyc_taxi_gold_crawler</code>	<code>Ready</code>	-		July 1, 2026 at 05:36:54	View log L2	1 created
<code>nyc_taxi_silver_crawler</code>	<code>Ready</code>	-		July 1, 2026 at 05:36:54	View log L2	1 created

Figure 9: AWS Glue Crawlers — All three crawlers (`bronze`, `silver`, `gold`) in `READY` state with successful runs



The screenshot shows the AWS Glue console for the `nyc_taxi_bronze_crawler`. The crawler properties are listed, and a table of crawler runs is shown below. The table has columns for Start time (UTC), End time (UTC), Current/last duration, Status, DPU hours, and Table changes. Three runs are listed, all with a `Completed` status.

Start time (UTC)	End time (UTC)	Current/last duration	Status	DPU hours	Table changes
July 1, 2026 at 05:36:53	July 1, 2026 at 05:37:35	41 s	Completed	0.118	1 table change, 1 partition change
July 1, 2026 at 05:02:20	July 1, 2026 at 05:03:02	41 s	Completed	0.118	-
July 1, 2026 at 04:25:56	July 1, 2026 at 04:26:36	40 s	Completed	0.116	1 table change, 1 partition change

Figure 10: Glue Crawler Details — *nyc_taxi_bronze_crawler* showing multiple successful runs

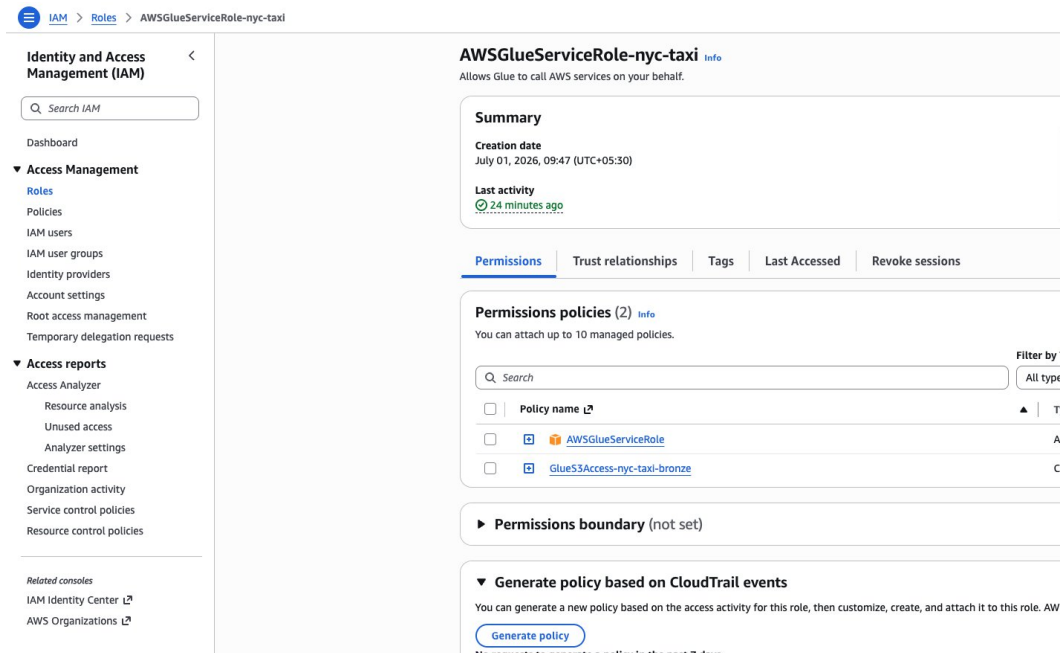


Figure 11: IAM Role — *AWSGlueServiceRole-nyc-taxi* with *GlueS3Access-nyc-taxi-bronze* policy

9. AMAZON REDSHIFT SERVERLESS (Loaded after Glue)

- **Implementation:**
 - Workgroup: ``nyc-taxi-workgroup`` with namespace ``nyc-taxi-namespace``
 - Schema: ``nyc_taxi`` with two tables (``daily_summary`` and ``hourly_demand``)
 - Idempotent loading pattern: ``DELETE`` existing partition → ``COPY`` from S3 → ``UPDATE`` to backfill partition columns
 - Uses IAM role for secure S3 access (no hardcoded credentials)
 - Triggered by Airflow **after** Glue crawlers complete

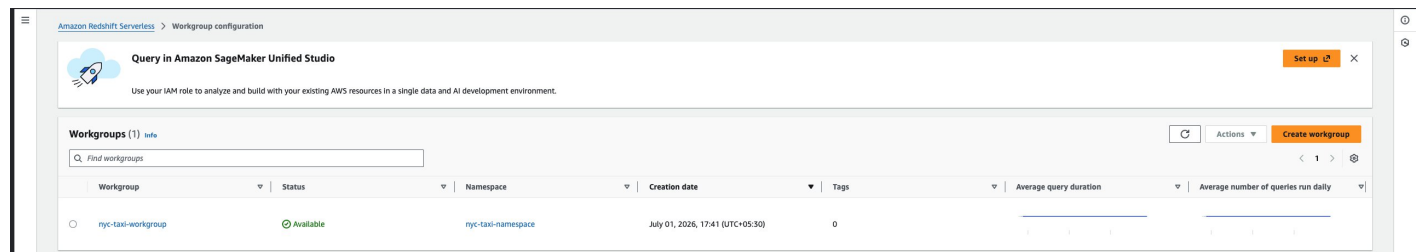


Figure 12: Redshift Serverless Workgroup — *nyc-taxi-workgroup* successfully created and available

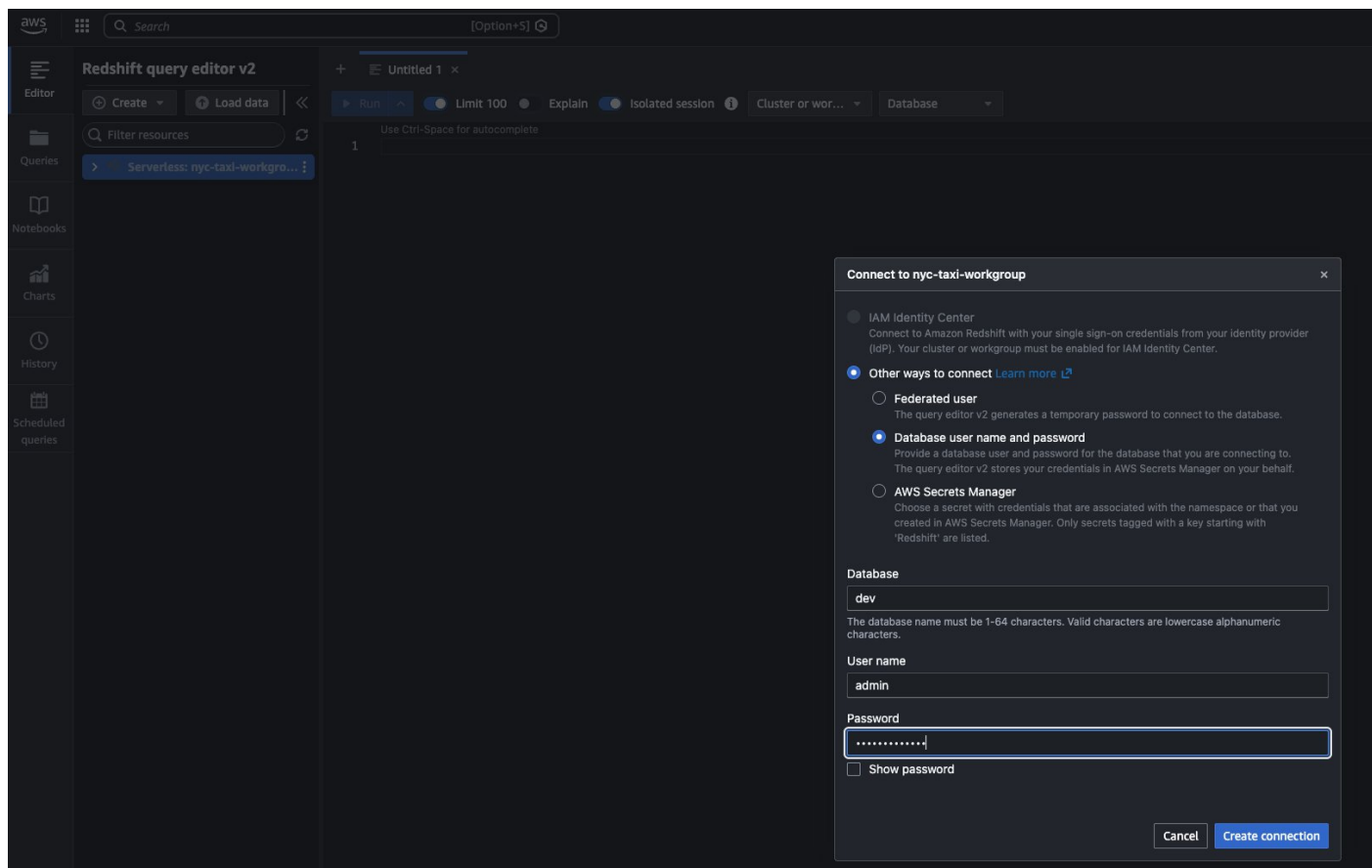


Figure 13: Redshift Connection — Connected to dev database using admin credentials via Query Editor v2

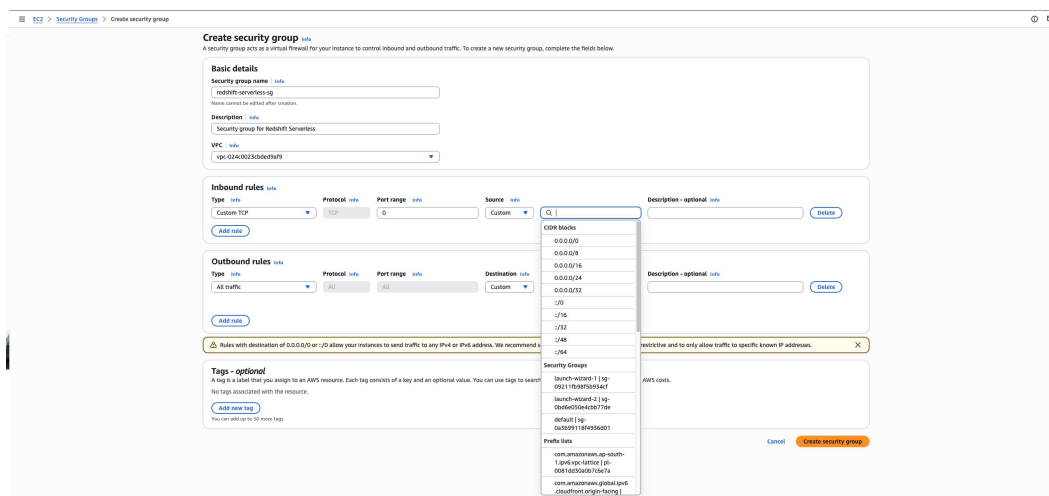


Figure 14: Security Group — redshift-serverless-sg created with proper inbound/outbound rules

```
(.venv) himanshu@Himanshus-MacBook-Pro 10.nyc_taxi_aws_data_engineering % >...
try:
    conn = psycopg2.connect(
        host=config['redshift']['host'],
        port=int(config['redshift']['port']),
        dbname=config['redshift']['dbname'],
        user=config['redshift']['user'],
        password=config['redshift']['password'],
        connect_timeout=10
    )
    print(✅ Successfully connected to Redshift!)
    conn.close()
except Exception as e:
    print(f❌ Failed: {e}')
"
```

```
✅ Successfully connected to Redshift!
```

```
(.venv) himanshu@Himanshus-MacBook-Pro 10.nyc_taxi_aws_data_engineering % docker compose exec airflow-worker python -c "
from warehouse.redshift_utils import create_redshift_schema_and_tables

create_redshift_schema_and_tables()
print(✅ Tables created successfully in Redshift!)"
"
```

```
✅ Tables created successfully in Redshift!
```

```
(.venv) himanshu@Himanshus-MacBook-Pro 10.nyc_taxi_aws_data_engineering %
```

Figure 15: Redshift Connection and table creation test on cli

```
(.venv) himanshu@Himanshus-MacBook-Pro 10.nyc_taxi_aws_data_engineering % >...
    MIN(pickup_year) as min_year,
    MAX(pickup_year) as max_year,
    MIN(pickup_month) as min_month,
    MAX(pickup_month) as max_month
FROM nyc_taxi.daily_summary
WHERE pickup_year = 2024 AND pickup_month = 1
UNION ALL
SELECT
    'hourly_demand' as table_name,
    COUNT(*) as row_count,
    MIN(pickup_year),
    MAX(pickup_year),
    MIN(pickup_month),
    MAX(pickup_month)
FROM nyc_taxi.hourly_demand
WHERE pickup_year = 2024 AND pickup_month = 1;
''')

for row in cur.fetchall():
    print(row)

cur.close()
conn.close()
print('\n✅ Verification completed')
"
```

```
=== Verification: Redshift Data (2024-01) ===
('hourly_demand', 168, 2024, 2024, 1, 1)
('daily_summary', 7, 2024, 2024, 1, 1)
```

```
✅ Verification completed
```

Figure 16: Redshift Data Verification — Python script confirming daily_summary (7 rows) and hourly_demand (168 rows) for January 2024

10. AMAZON ATHENA — Querying Gold Layer via Glue Catalog

After Glue crawlers complete, the Gold layer becomes queryable through Amazon Athena using the Glue Catalog. This serves as an additional verification layer.

The screenshot displays the Amazon Athena Query Editor interface. The top navigation bar includes 'Editor', 'Recent queries', 'Saved queries', and 'Query settings'. A notification banner states: 'Athena now supports typeshed code suggestions to speed up SQL query development. Typeshed suggestions are turned on by default. You can change this setting in query editor preferences.' The left sidebar shows the 'Data' section with 'Data source' set to 'AwsGlueCatalog', 'Catalog' set to 'nyctaxi', 'Database' set to 'nyctaxi_db', and a 'Tables and views' section listing various tables like 'bronze_nyc_taxi', 'gold_nyc_taxi', and 'silver_nyc_taxi'. The main query editor shows a SQL query: 'SELECT * FROM "nyctaxi"."gold_nyc_taxi" limit 10;'. Below the query editor, the 'Query results' section shows 'Results (10)' with a table of data. The table has columns: 'pickup_datetime', 'total_trips', 'total_revenue', 'avg_trip_duration_min', 'avg_trip_amount', 'avg_passenger_count', 'pickup_hour', 'partition_0', 'pickup_year', and 'pickup_month'. The results show data for both 'daily_summary' and 'hourly_demand' partitions.

#	pickup_datetime	total_trips	total_revenue	avg_trip_duration_min	avg_trip_amount	avg_passenger_count	pickup_hour	partition_0	pickup_year	pickup_month
1	1	306420	8408507.019999234	14.5459269949728	3.487520907732843	1.4265909535931074		daily_summary	2024	1
2	2	372450	1.0785521489998817E7	15.87163686408885	3.6137171224523416	1.3180238958249429		daily_summary	2024	1
3	3	423939	1.1814320123999176E7	16.19478939422564	3.501889533712359	1.3053664520509457		daily_summary	2024	1
4	4	458815	1.2638954999498527	16.254937173071547	3.501225864257325	1.2994143237950564		daily_summary	2024	1
5	5	501404	1.1055586179099476E7	16.41011814014761	3.52862604095946	1.310581504462079		daily_summary	2024	1
6	6	578057	1.02039555999970E7	15.91926100655988	3.4588091051153873	1.3642111542826278		daily_summary	2024	1
7	7	384284	9702274.89999951	14.89307678333348	3.216229881472127	1.45121261764736496		daily_summary	2024	1
8	1	17461	487919.500000008	14.51866481541964		0		hourly_demand	2024	1
9	1	15886	310446.5199999998	13.45062449318153		1		hourly_demand	2024	1

Figure 17: Amazon Athena Query — Successfully querying gold_nyc_taxi table showing both daily_summary and hourly_demand partitions

```

23 @dag( @Himanshu Patidar
24     dag_id="nyc_taxi_data_pipeline",
25     schedule="@monthly",
26     start_date=datetime( year= 2024, month= 1, day: 1),
27     catchup=False,
28     tags=["nyc", "taxi", "bronze", "silver", "gold", "glue", "athena", "redshift"],
29     params={"year": 2024, "month": 2}
30 )
31 def nyc_taxi_data_pipeline():
32
33     @task @Himanshu Patidar
34     def bronze_ingestion(**context):
35         params = context["params"]
36         year = params.get("year", 2024)
37         month = params.get("month", 1)
38
39         logger.info("=== NYC Taxi Bronze Ingestion ===")
40         s3_key = download_and_upload_to_bronze(year, month, AWS_BUCKET_NAME)
41         logger.info(f"Bronze file landed at: s3://{AWS_BUCKET_NAME}/{s3_key}")
42         return {"year": year, "month": month}
43
44     @task @Himanshu Patidar
45     def silver_transformation(bronze_result: dict):
46         year = bronze_result["year"]
47         month = bronze_result["month"]
48
49         process_silver_layer(year, month, AWS_BUCKET_NAME, AWS_BUCKET_NAME)
50         return {"year": year, "month": month}
51
52     @task @Himanshu Patidar
53     def data_quality_check(silver_result: dict):
54
55         from pyspark.sql import SparkSession
56         from pyspark.sql.functions import col
57
58         year = silver_result["year"]
59         month = silver_result["month"]
60
61         spark = (
62             SparkSession.builder
63             .appName("Airflow_Data_Quality")
64             .config("spark.jars.packages",
65                  "org.apache.hadoop:hadoop-aws:3.4.2,com.amazonaws:aws-java-sdk-bundle:1.12.787")
66             .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
67             .config("spark.hadoop.fs.s3a.aws.credentials.provider",
68                  "com.amazonaws.auth.DefaultAWSCredentialsProviderChain")
69             .config("spark.hadoop.fs.s3a.multipart.enabled", "false")
70             .getOrCreate()
71         )
72
73         silver_path = f"s3a://{AWS_BUCKET_NAME}/silver/nyc_taxi/"

```

Figure 18: Actual Project Code

11. END-TO-END PIPELINE EXECUTION FLOW

- **Step 1 (Orchestration):** Airflow DAG triggered with `year=2024, month=1` parameters
- **Step 2 (Bronze):** `bronze\_ingestion` task downloads Parquet from CloudFront → uploads to S3 (idempotent)
- **Step 3 (Silver):** `silver\_transformation` task reads Bronze, cleans data, derives columns, runs quality checks
- **Step 4 (Gold):** `gold\_aggregation` task creates daily_summary and hourly_demand for the month

- **Step 5 (Glue):** `setup_glue_infrastructure` + `run_glue_crawlers` tasks catalog all three layers in Glue
- **Step 6 (Redshift):** `setup_redshift_warehouse` + `load_redshift_warehouse` tasks create schema/tables and COPY Gold data (idempotent)
- **Step 7 (Verification):** Redshift Query Editor + Athena confirm correct row counts and data quality

12. KEY IMPLEMENTATION CHALLENGES & SOLUTIONS

- **Challenge 1: Airflow 3.x Migration**
 - **Solution:** Updated all imports to use `from airflow.sdk import dag, task`. Removed deprecated `sys.path.insert` and duplicate DAG registration logic.
- **Challenge 2: Redshift COPY with Spark Partitioned Parquet**
 - **Solution:** Spark's `partitionBy()` removes partition columns from physical files. Implemented `DELETE` + `COPY` (only non-partition columns) + `UPDATE` to backfill `pickup_year` and `pickup_month`.
- **Challenge 3: Data Quality Null/NaN Handling**
 - **Solution:** Created `_is_null_or_nan()` helper that checks column data type before calling `isnan()` to avoid `AnalysisException` on non-numeric columns.
- **Challenge 4: Idempotent Monthly Pipeline Runs**
 - **Solution:** Bronze layer checks S3 object existence before download. Redshift uses `DELETE` before `COPY` for the specific year/month partition.

13. SKILLS DEMONSTRATED

- Modern Data Engineering with **Medallion Architecture** on AWS
- **Apache Airflow 3.x** (TaskFlow API, parameterization, Docker + Celery deployment)
- **PySpark** for large-scale data cleaning, transformation, and aggregation
- **AWS Glue** for automated cataloging and schema evolution
- **Amazon Redshift Serverless** data loading best practices (COPY + idempotency patterns)
- **Data Quality** frameworks and validation at scale
- Production-grade pipeline design with **idempotency**, error handling, and observability
- Infrastructure as Code mindset for reproducible environments